MAC flooding and port security

Follow-along: play the attacker, then the defender

Overview

Definition: MAC learning table

The table a switch uses to decide where to send a frame. Each row maps one Ethernet address to the one port that address was last seen sending from, so the table is written by the source address of arriving frames and read by the destination address. It holds a fixed number of rows.

Definition: Flooding an unknown destination

What a switch does with a unicast frame whose destination address is not in its table: it sends a copy out of every other port, because it has no way to know which one is right. The frame still reaches its destination. It also reaches everyone else.

A switch only knows where a host is because it saw that host send something. That knowledge lives in a **MAC learning table** of fixed size, and a MAC flooding attack is an attempt to spend it: the attacker sends frames from thousands of invented source addresses, each one costing a row, until the switch has no room left for the addresses it actually needs. An address the switch has forgotten is an address it cannot look up, and **flooding an unknown destination** means every frame sent to that address is copied to every port. Whoever is on those ports gets to read traffic that was never addressed to them.

That is the textbook account, and on the switch in this lab half of it is wrong. You will flood the table until it is completely full, watch tens of thousands of rows be discarded, and find that the host on the port next to you loses nothing at all, because Open vSwitch does not discard rows in the order the classic account assumes. What the flood does reach is the one port that carries more addresses than the others, which in this network, and in most real ones, is the link to the rest of the site. Part 2 closes it twice: once by writing the forwarding decision somewhere a flood cannot touch, and once by refusing the forged addresses at the port they arrive on.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab   # first time
cd ~/MiniLab && git pull                                      # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

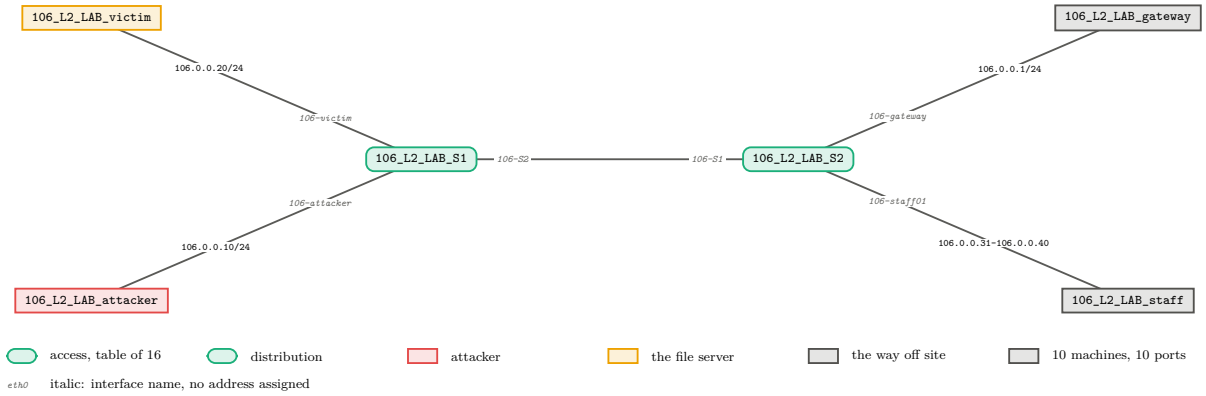
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one lab per row. Select **MAC flooding and port security** to open its actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: how full the switch's address table is, which port is holding how much of it, and whether the victim's traffic is reaching the attacker. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** asks which node (attacker, victim, gateway, staff, S1, S2) and prints the `docker exec -it ...` line to paste into your own terminal; each step below names the node it runs on.
- **Reset to baseline** stops the attack and drops your defence, returning the lab to its clean Part 1 state without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Definition: Access port and uplink

An access port has a single host on the end of it, so the switch learns one address there. An uplink joins one switch to another, so the switch learns the address of every host on the far side through that single port. Both are ordinary ports; the difference is how many addresses each accounts for.

Everything is one subnet, `106.0.0.0/24`, spread across two switches. S1 is the switch under attack, and its **access port and uplink** carry very different loads: the victim and the attacker are one address each, while the uplink to S2 accounts for eleven, being the gateway and the ten staff machines behind it. S1's table has been set to hold 16 addresses. A real switch holds thousands, and the small number here is what makes the lab fill in milliseconds and measure the same way twice.

Role	Address	Container	Port on the switch
Access switch	–	`106_L2_LAB_S1`	table limited to 16 entries
Distribution switch	–	`106_L2_LAB_S2`	table left at 2048
Victim (file server)	`106.0.0.20`	`106_L2_LAB_victim`	`106-victim` on S1
Attacker	`106.0.0.10`	`106_L2_LAB_attacker`	`106-attacker` on S1
Gateway	`106.0.0.1`	`106_L2_LAB_gateway`	`106-gateway` on S2
Staff machines	`106.0.0.31-.40`	`106_L2_LAB_staff`	`106-staff01` to `106-staff10` on S2
Uplink	–	–	`106-S2` on S1, `106-S1` on S2

The victim is the lab segment's file server. The ten staff machines poll it constantly, which is why their addresses keep arriving at S1 through the uplink, and the victim sends its own traffic out through the gateway. That second path, victim to gateway, is the one the attacker wants to read.

On the attacker you have `tcpdump` and `scapy` from the base image, and one tool built for this lab:

- `mac-flood <interface> <frames> <pps>` sends frames from random source addresses. It takes no defaults: you name the interface, how many frames (0 means until you stop it), and how many per second.

Assessment

There are 15 questions, placed at the step they belong to. Answer them in a separate document, numbered to match. Several ask for a number you read off your own screen, so record it as you go rather than reconstructing it afterwards; your figures will not match another student's exactly, and they are not meant to.

Part 1: fill the table

Read the table before you touch it

Everything in this lab is one number moving, so start by knowing where it started. Open a shell on S1 and ask the switch how full its table is and what is in it:

```
# 106_L2_LAB_S1: the access switch under attack
ovs-appctl fdb/stats-show br0
ovs-appctl fdb/show br0
```

`fdb/show` prints one row per address, and the first column is a port *number* rather than a name. Map the numbers to names with:

```
# 106_L2_LAB_S1: which port number is which
ovs-ofctl show br0
```

Question 1. Using the two commands above, give the number of addresses S1 currently holds on each of its three ports (`106-victim`, `106-attacker` and the uplink `106-S2`). Give the total, the table's limit, and how many rows are still free.

Send one frame and watch it cost a row

Before flooding anything, send a single frame and see what the switch does with it. Build it by hand so that every field is yours: a source address the switch has never seen, and a destination address belonging to nobody.

```
# 106_L2_LAB_attacker: one frame, one invented source address
python3 -c "from scapy.all import Ether, sendp;
    sendp(Ether(src='02:00:00:00:00:01', dst='02:00:00:00:00:99'), iface='106-S1')"
```

Then look for it on S1:

```
# 106_L2_LAB_S1: did that address cost a row, and on which port?
ovs-appctl fdb/show br0 | grep 02:00:00:00:00:01
```

Question 2. The frame you sent has two addresses in it and they were treated differently. Say which of the two put a new row in S1's table and why that one, then say what S1 did with the frame itself, given that the destination address is in nobody's table.

Turn the flood on

One frame costs one row, so 16 frames would fill this table. Send far more than that, and keep sending, because the table has to stay full for the rest of Part 1. Leave this running in a terminal of its own:

```
# 106_L2_LAB_attacker: 5000 frames a second, until you stop it with Ctrl-C
mac-flood 106-S1 0 5000
```

Now go back to S1 and read the same two commands as before.

Question 3. Record, while the flood is running: the number of entries in use and the limit, the "Total number of evicted MAC entries" count, and the number of addresses now held on each of S1's three ports. Take the port counts twice, about ten seconds apart, and say whether they are still moving.

The port that did not lose anything

Look at the victim's port in the numbers you just recorded, and compare it with what you recorded before the flood started.

Question 4. Tens of thousands of rows have been discarded, and the victim's access port has kept its entry throughout. Use your own two sets of port counts to rule out the rule "when the table is full, discard the row that has gone longest without being used, wherever it is". State what your numbers show is being discarded instead.

Definition: Eviction, and the fair share rule

Discarding a row to make space for a new one. Open vSwitch does not pick the oldest row in the table. It picks the port currently holding the most rows, and discards that port's least recently used row, "so that there is a degree of fairness, that is, each port is entitled to its fair share of MAC entries" (`lib/mac-learning.c`). A port holding one address is never the port holding the most, so its row is never taken. Worked through: with a 16-row table, a victim port holding 1 and an attacker flooding, the attacker's port grows until it is the largest, and from that point every new forged address costs the attacker one of its own rows.

Eviction by fair share is why the attack you have just run does nothing to the host beside you, and it is worth being precise about what the rule is: a decision Open vSwitch's authors made, not a rule of Ethernet. Before carrying anything you measure here to another switch, find out what that switch does when its table fills.

Question 5. Apply the rule to this lab's own numbers. The table holds 16 rows and the victim's port holds 1 of them and is never the largest, so two ports are competing for what is left. Work out how many rows each of those two settles at, and compare your answer with what you measured in Question 3.

Question 6. Among the eleven addresses reachable through the uplink, the ten staff machines send to the file server ten times a second, while the gateway only sends when it answers something. Given that eviction takes a port's *least recently used* row, say which of the eleven is taken first and why. Check your answer: get the gateway's address from its own container and look for it in S1's table.

The gateway's address is on its interface facing S2:

```
# 106_L2_LAB_gateway: read this address, you need it for the capture below
cat /sys/class/net/106-S2/address
```

Read someone else's traffic

The attacker is on an access port. Nothing addressed to the gateway should ever arrive there. Capture exactly that, substituting the address you just read:

```
# 106_L2_LAB_attacker: frames addressed to the gateway, arriving on the attacker's
    own port
tcpdump -n -e -i 106-S1 'ether dst <gateway address>'
```

With that running and the flood still going, make the victim talk to the gateway:

```
# 106_L2_LAB_victim: ten frames to the gateway
ping -c 10 106.0.0.1
```

Question 7. How many of the victim's ten frames to the gateway arrived on the attacker's port? Quote one captured line, and identify which host sent it and which host it was addressed to.

Question 8. Your capture holds the victim's frames to the gateway and none of the gateway's

replies to the victim, although both cross S1. Explain the asymmetry in terms of where each frame's *destination* address sits in S1's table.

Question 9. The victim's ping succeeded the whole time: ten sent, ten answered. Explain why a switch that has forgotten where the gateway is still delivers to it, and say what that costs the network that a correct table would not.

Question 10. Your `mac-flood` run reports the rate it achieved when you stop it, and the switch here holds 16 addresses. Suppose instead a switch holding 32,768, with the same three ports and the same eleven addresses reachable through the uplink. At the rate you measured, how long would filling it take, and how many frames is that? Then apply the fair share rule to that full table: work out how many rows the attacker's port ends up holding, say which port every eviction after that comes from, and say whether the small table in this lab is what made the attack work.

Stop the flood before you go on, and put the lab back to its starting state with the **Reset to baseline** action, so Part 2 begins from a table that is not already full.

Part 2: keep the table out of the decision

Two things went wrong, and they are worth separating because the fixes are different. The switch lost the gateway's address, which is why the victim's frames were flooded; and it lost that address because a host on an access port was allowed to introduce thousands of addresses nobody asked for. You will fix each in turn. The first takes the forwarding decision for one address out of the learning table altogether, so it no longer matters whether the table remembers the gateway. The second stops the forged addresses at the port they arrive on, so the table never fills in the first place.

The pin that does not hold

Open vSwitch offers what looks like exactly the right tool. `ovs-appctl fdb/add` writes a row by hand and marks it `static`, and `fdb/show` prints `static` where it would otherwise print an age. It does not survive this attack. A static row is exempt from ageing but not from eviction: the ageing code skips static rows explicitly, and the eviction code has no equivalent check (`lib/mac-learning.c`). Try it and watch it go, with the flood running again:

```
# 106_L2_LAB_S1: pin the gateway to the uplink by hand, then watch for a few
    seconds
ovs-appctl fdb/add br0 106-S2 0 <gateway address>
ovs-appctl fdb/show br0 | grep -i <gateway address>
```

Question 11. A row's place in the eviction order is refreshed each time that address is learned again, and nothing ever re-learns a row you added by hand. Using those two facts and the fair share rule, explain why a static row is not merely unprotected but is among the first rows the flood takes.

Defence 1: decide it in the flow table

Definition: Flow table

A table of rules the switch consults before its ordinary switching behaviour runs. Each rule matches on fields of the frame and says what to do with it. The rule that produces normal switching, which includes the learning-table lookup, is itself just an entry in this table, at the lowest priority; a higher-priority rule matching the same frame is used instead. The flow table is separate storage from the MAC learning table, and nothing a flood does can evict an entry from it.

Write one rule in the **flow table** saying where frames addressed to the gateway go. S1 then stops asking the learning table a question the flood has made it unable to answer. You need the uplink's port number, which is the number beside `106-S2` in `ovs-ofctl show br0`:

```
# 106_L2_LAB_S1: send anything addressed to the gateway out of the uplink,
    whatever the table says
ovs-ofctl add-flow br0 "priority=200,dl_dst=<gateway
    address>,actions=output:<uplink port number>"
```

Question 12. Before you test it: the flood is still running and the table is still full. Predict whether the victim can still reach `106.0.0.37`, one of the staff machines, and give your reasoning. Then test it with a ping from the victim and report what happened. The rule you added names one destination address; say what that means for whether the attacker can still read the victim's traffic to `106.0.0.37`.

Check the leak with the same `tcpdump` and `ping` pair you used in Part 1, then look at the table again with `fdb/stats-show`. The victim's frames to the gateway stop reaching the attacker; the table is still full and the uplink is still squeezed.

Defence 2: refuse the addresses at the door

Definition: Port security

Limiting which source addresses a switch will accept on a given port. A port with one host behind it needs exactly one, so anything else arriving there is either a mistake or an attack, and dropping it costs the legitimate host nothing. Dropping happens on arrival, before the switch learns anything from the frame, so a refused address never reaches the table.

Port security fixes the cause rather than the symptom. Two rules: the first permits the attacker's real address on its port and hands those frames to normal switching, and the second, at a lower priority, drops everything else arriving on that port. Read the attacker's real address and its port number first, then write both rules:

```
# 106_L2_LAB_attacker: the one source address this port is entitled to use
cat /sys/class/net/106-S1/address
```

```
# 106_L2_LAB_S1: permit that one address, drop every other source on the same port
ovs-ofctl add-flow br0 "priority=300,in_port=<attacker port
    number>,dl_src=<attacker address>,actions=NORMAL"
ovs-ofctl add-flow br0 "priority=100,in_port=<attacker port number>,actions=drop"
```

Read the table now, with the flood still running. The forged addresses have stopped arriving, but the ones already learned are still there and will sit for the full ageing time. Clear them:

```
# 106_L2_LAB_S1: drop the rows the flood already put in, then read the table again
ovs-appctl fdb/flush br0
ovs-appctl fdb/stats-show br0
```

Question 13. Between adding the two rules and running `fdb/flush`, the table was still full and the uplink was still short of addresses, even though no new forged address was arriving. Explain what those rows were, and why the rules alone did not remove them.

Question 14. A single rule, `priority=100,in_port=<attacker port number>,actions=drop`, would also stop the flood dead. Name what it costs that the two-rule version does not, and give an observation you could make from the attacker's container that tells the two apart.

Question 15. Compare the two defences. Name one thing each protects that the other does not, then say which one you would deploy across 200 access ports on a real switch, and what makes it the more practical of the two at that scale.

Checks: what "done" means

Run the **Status** action after each stage. It reads the same numbers you have been reading by hand, and the three that matter are:

- **The uplink's share of the table.** 11 addresses at baseline; fewer while the flood is landing; back to 11 once port security is in place, with the flood still running.
- **Whether the gateway's address is present.** Present at baseline, missing under the attack.
- **The leak oracle.** Status counts frames arriving on the attacker's port that are addressed to the gateway. `CONTAINED` before the attack and after either defence, `LEAKING` in between.

A defence that reports `CONTAINED` has only half passed. The other half is that honest traffic still works: the victim still reaches both the gateway and the staff machines, and the host on the secured port still reaches the gateway itself. Status prints all three.

Safety note

This lab runs on its own two Open vSwitch bridges inside containers, with no route off the host. `mac-flood` sends on the interface you name and nowhere else, and the only interface the attacker has is the veth into S1. Nothing here reaches a network you did not spawn, and no step needs `sudo`.

Tear the lab down

When you have finished, remove the lab: pick **Teardown** in the TUI, or run `scripts/teardown.sh` directly. It deletes this lab's six containers and nothing else.